

UNIVERSIDAD
DE GRANADA

Artificial Bee Colony

Práctica opcional

Pablo Linari Pérez

DNI: 75571079W
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del algoritmo 3
1.1 Funcionamiento del algoritmo 3
1.2 Parámetros del algoritmo 3
1.3 Ecuaciones 4
1.4 Etapas del algoritmo 4
1.4.1 Etapa 1: Inicialización 4
1.4.2 Etapa 2: Fase de abejas empleadas 4
1.4.3 Etapa 3: Fase de abejas observadoras 5
1.4.4 Etapa 4: Fase de abeja exploradora 5
1.5 Pseudocódigo 5
2 Adaptacion al problema del portfolio 6
2.0.1 Comparación de resultados 6
3 Mejoras del algoritmo 9
3.1 Hibridación con búsqueda local (Solis-Wets) 9
3.1.1 Búsqueda local: Solis-Wets 9
3.1.2 Integración en el ABC 10
3.2 Hibridación con Differential Evolution 10
3.2.1 Hibridación completa con DE 10
3.2.2 Uso del operador de DE para generación de candidatos 11
3.3 Comparación de las hibridaciones 12
4 Propuestas de mejora 14
4.1 ABC guiado por la mejor solución global Gbest-guided ABC 14
4.1.1 Regla de generación del candidato 14
4.2 ABC guiado por gbest con límite de dinámico 15
4.2.1 Medida de diversidad 15
4.2.2 Límite dinámico 16
4.3 Comparación de resultados 16

1 Descripción del algoritmo

El algoritmo **Artificial Bee Colony** (ABC), propuesto por Dervis Karaboga en 2007, es una metaheurística poblacional inspirada en el comportamiento de búsqueda de alimento de las colonias de abejas. La idea del algoritmo es que un conjunto de abejas explora el espacio de búsqueda en el que hay una serie de fuentes de alimentos, intensificando la exploración alrededor de las fuentes más prometedoras y abandonando aquellas que se agotan.

En el contexto de este problema cada **fuente de alimento** representa una solución candidata, y la cantidad de néctar de una fuente se corresponde con el valor de fitness de esa solución. El objetivo del algoritmo es, por tanto, encontrar la fuente con mayor cantidad de néctar, es decir, la solución que maximiza la función objetivo.

==Explicación de los componentes del algoritmo

La colonia se divide en tres tipos de abejas, cada uno con un papel distinto dentro de la búsqueda:

- **Abejas obreras (worker bees)**: cada una está asociada a una fuente de alimento concreta y se encarga de explotarla buscando soluciones mejores en su vecindario.
- **Abejas observadoras (onlooker bees)**: esperan en la colmena y eligen qué fuentes explotar en función de la información que comparten las obreras. Las fuentes con más néctar tienen más probabilidad de ser elegidas, esto hace que se concentre la búsqueda en las funetes de alimento más prometedoras.
- **Abejas exploradoras (scout bees)**: cuando una fuente se agota se abandona, y la abeja obrera que trabajaba sobre esta se convierte en exploradora, buscando una fuente de alimento nueva.

El número de fuentes de alimento, según se indica en el artículo, es la mitad del tamaño total de la colonia, y coincide tanto con el número de abejas trabajadoras como con el de abejas observadoras ya que cada fuente tiene una trabajadora asociada.

Esta forma de inicializar la población es la propuesta en el artículo , no obstante veremos que se puede modificar este parámetro para cambiar el comportamiento del algoritmo aunque no sea el parámetro que más influya.

1.1 Funcionamiento del algoritmo

1.2 Parámetros del algoritmo

El comportamiento del algoritmo se controla mediante los siguientes parámetros:

- **colonySize**: tamaño total de la colonia. El número de fuentes de alimento se calcula como $SN = \frac{\text{colonySize}}{2}$. En la implementación se usa un valor de 50, por lo que $SN = 25$.
- **Limit** : número máximo de ciclos que una fuente puede permanecer sin mejorar antes de ser abandonada. Controla cuanto se pueden explotar las soluciones.
- **maxevals**: número máximo de evaluaciones de la función objetivo, que actúa como criterio de parada.

1.3 Ecuaciones

El algoritmo usa dos ecuaciones . La primera genera una solución candidata v_i a partir de una fuente existente x_i, perturbando una única componente j elegida al azar:

$$v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj})$$

donde $k \neq i$ es otra fuente vecina elegida aleatoriamente y ϕ_{ij} es un número aleatorio en el intervalo $[-1, 1]$. El resto de componentes de v_i se copian directamente de x_i.La nueva solución se acerca o se aleja de una fuente vecina en una sola dimensión.

Para entenderlo mejor , este movimiento en el problema del portfolio equivale a tomar una solución i , y sumar o resar a una empresa de dicha solución un porcentaje de la misma empresa pero de la otra solución escogida k.De esta forma nos acercaríamos o alejaríamos de la solución escogida en dicha componente.

La segunda ecuación calcula la probabilidad de que cada fuente sea seleccionada por una observadora, de forma proporcional a su néctar:

$$p_i = \frac{\text{fit}_i}{\sum_{n=1}^{SN} \text{fit}_n}$$

Cuanto mayor es el fitness de una fuente, mayor es su probabilidad de ser explotada por las observadoras.

1.4 Etapas del algoritmo

La ejecución se divide en 4 fases claras donde se identifica el comportamiento de cada abeja con su rol.

1.4.1 Etapa 1: Inicialización

Se generan SN fuentes de alimento de forma aleatoria, asegurando que cada una cumple las restricciones del problema (en el caso del portfolio se comprueban las restricciones necesarias). Se evalúa el fitness de cada una y se memoriza la mejor solución encontrada hasta el momento. Además, se inicializa a 0 un contador de *trials* por cada fuente, que registra cuántos ciclos lleva sin mejorar.

1.4.2 Etapa 2: Fase de abejas empleadas

Cada abeja empleada explota su propia fuente x_i. Para ello genera una solución candidata v_i aplicando la ecuación (2.2) y la evalúa. A continuación se aplica una selección greedy: si la candidata es mejor que la fuente original, la sustituye y el contador de *trials* de esa fuente se reinicia a 0, en caso contrario, la fuente se mantiene y su contador de *trials* se incrementa. Esta fase genera por cada fuente de alimento una posible solución mejor usando la ecuación previamente vista, si esta solución es mejor será la que se use como base para la explotación.

1.4.3 Etapa 3: Fase de abejas observadoras

Esta etapa consiste en el «baile de las obreras» donde cada obrera vuelve a la colmena e intentará convencer a las observadoras de elegir su fuente de alimento , las observadoras eligirán con mayor probabilidad la fuente que más nectar tenga.Los pasos que se siguen una vez la obrera vuelve a la colmena son los siguientes. Primero se calculan las probabilidades de selección de cada fuente con la segunda ecuación . Después, se reparten SN observadoras: cada una elige una fuente mediante selección por ruleta ,(las fuentes con más néctar tienen más probabilidad) y la explota igual que una empleada, usando la misma ecuación y aplicando de nuevo la selección greedy. Como las mejores fuentes tienden a ser elegidas varias veces, esta fase intensifica la búsqueda en las regiones más prometedoras del espacio. Es decir que podemos tener varias abejas observadoras en la misma fuente de alimento, esto ocurrirá sobre todo en las fuente más prometedoras donde como es lógico, interesa intensificar la búsqueda.

1.4.4 Etapa 4: Fase de abeja exploradora

Al final de cada ciclo se comprueba si alguna fuente se ha agotado. Se localiza la fuente con mayor número de *trials* acumulados y, si este supera el *Limit*, dicha fuente se abandona, se sustituye y la abeja exploradora buscaría una nueva, que a modo práctico consiste en crear una nueva solución aleatoria,se evalúa la nueva solución y su contador de *trials* vuelve a 0. Se permite como máximo una abeja exploradora por ciclo. Esta fase introduce diversificación y evita que el algoritmo quede estancado en óptimos locales.

En las tres fases se actualiza siempre la mejor solución global encontrada, que es la que finalmente devuelve el algoritmo.

1.5 Pseudocódigo

```
funcion ABC(problema, maxevals):
  SN = colonySize / 2
  # --- Inicialización ---
  para i en 0..SN:
    fuentes[i]   = problema.createSolution()
    fitness[i]   = problema.fitness(fuentes[i])
    trials[i]    = 0
    actualizar_mejor(fuentes[i], fitness[i])

  # --- Ciclos de búsqueda ---
  mientras evals < maxevals:

    # Fase empleadas
    para i en 0..SN:
      candidata = produceCandidate(fuentes, i)
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)
```

```
    # Fase observadoras
    probs = computeProbabilities(fitness)
    para t en 0..SN:
      i = rouletteSelect(probs)
      candidata = produceCandidate(fuentes, i)
      si fitness(candidata) > fitness[i]:         # selección greedy
        fuentes[i] = candidata;  trials[i] = 0
      sino:
        trials[i] += 1
      actualizar_mejor(candidata)

    # Fase exploradora ( una por ciclo)
    peor = indice con mayor trials
    si trials[peor] >= trialLimit:
      fuentes[peor] = problema.createSolution()
      trials[peor]  = 0
      actualizar_mejor(fuentes[peor])

  devolver mejor_solucion
```

2 Adaptacion al problema del portfolio

La formulación original de ABC está pensada para la optimización de funciones numéricas sin restricciones, por lo que ha sido necesario realizar dos adaptaciones:

- **Reparación de soluciones**: la ecuación que se usa para explorar nuevas soluciones modifica una componente y, en general, rompe las restricciones del problema (los pesos deben sumar 1 y estar en $[\text{low}, \text{high}]$ o ser 0). Por ello, tras generar cada candidata se comprueba su validez y, si no es factible, se repara con `problem.fix()` antes de evaluarla.
- **Desplazamiento de las probabilidades**: la ecuación de cálculo de probabilidades asume cantidades de néctar no negativas, pero el fitness del portfolio puede ser negativo, de hecho en la práctica es siempre negativo. Para poder aplicar la ruleta, se desplazan todos los valores de fitness sumándoles una constante que deja el peor de ellos ligeramente por encima de 0, conservando las diferencias relativas entre fuentes.

Por lo que respecta a la implementación , la diferencia entre la implementación para el problema del portfolio y para el **cec2017** son las mencionadas previamente y el uso de las funciones de la clase **pPortfolio** , principalmente usadas para poder arreglar y generar soluciones con las restricciones del problema.

2.0.1 Comparación de resultados

En esta sección veremos los resultados que obtiene el ABC implementado sin ninguna modificación en el problema del portfolio, los algoritmos que se usan para la comparación serán: **Random Search , Greedy ,Local Search, AM-BEST , BMB y Enfriamiento simulado**.

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−4.558	−3.254	0.040	1	0.001
Random	−4.980	−3.378	0.052	10000	0.005
BL	−4.608	−3.115	0.020	1517.36	0.001
AM-BEST	−4.494	−3.084	0.032	10000	0.005
BMB	−4.566	−3.104	0.022	6990.3	0.001
ES	−4.594	−3.113	0.022	10000	0.003
ABC	−4.519	−3.139	0.025	10000	0.002

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−2.835	−3.691	0.092	1	0.001
Random	−3.408	−4.174	0.088	10000	0.02
BL	−3.118	−3.888	0.096	8984.76	0.01
AM-BEST	−2.814	−3.755	0.101	10000	0.044
BMB	−3.023	−3.837	0.092	10000	0.022
ES	−3.105	−3.895	0.095	10000	0.048
ABC	−3.036	−3.871	0.091	10000	0.011

Tabla 2: Resultados de S&P100 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−2.749	−3.776	0.013	1	0.001
Random	−3.638	−4.389	0.026	10000	0.253
BL	−3.517	−4.324	0.010	10000	0.220
AM-BEST	−3.220	−4.156	0.005	10000	0.456
BMB	−3.663	−4.450	0.013	10000	0.238
ES	−3.440	−4.343	0.012	10000	2.606
ABC	−3.584	−4.363	0.012	10000	0.180

Tabla 3: Resultados de S&P500 (Medias)

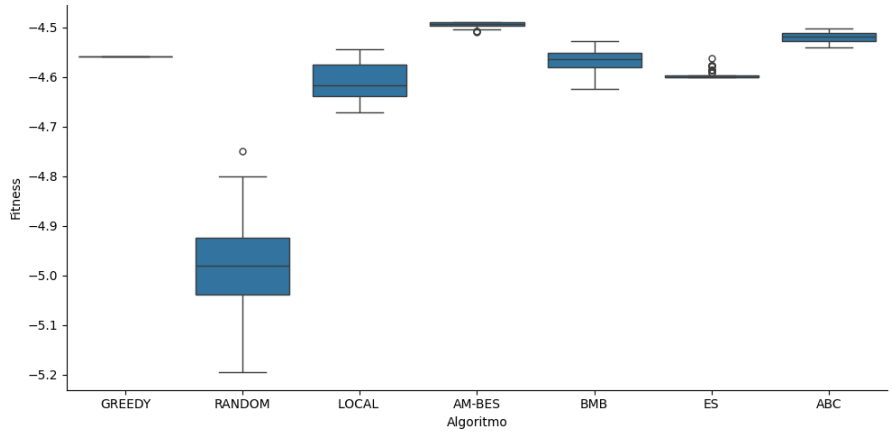


Figura 1: Boxplots Ibex35

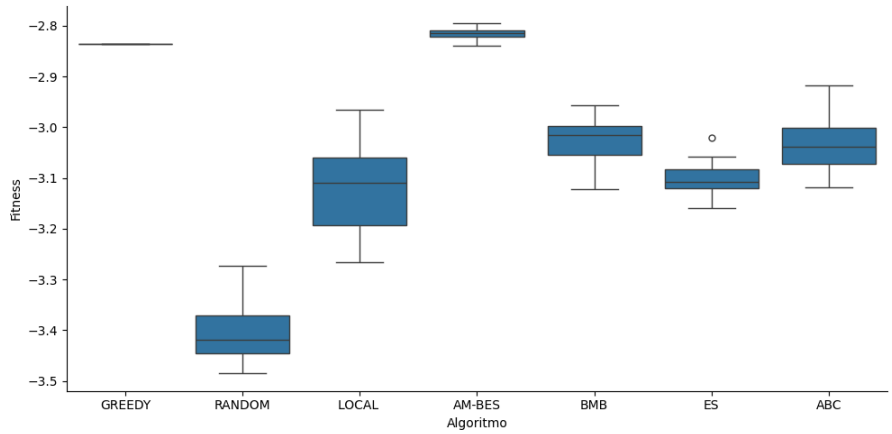


Figura 2: Boxplots S&P100

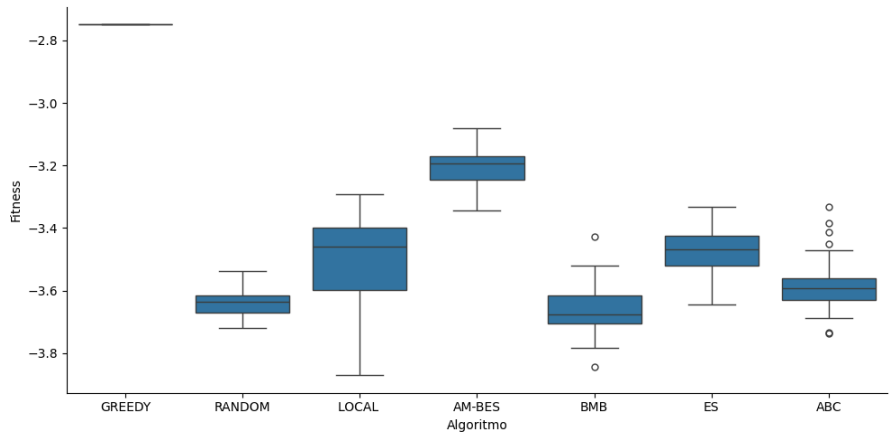


Figura 3: Boxplots S&P500

3 Mejoras del algoritmo

La mejoras del algoritmo se han implementado usando las funciones de **cec2017** para que la comparación pueda ser más efectiva y se pueda comparar en más problemas , no solo en el de prácticas

Veremos a lo largo de esta sección las diferentes mejoras que se han hecho sobre el algoritmo , se han planteado mejoras que hybridan el algoritmo de ABC con otros como el Solis-Wets o el Diferential evolution , y también se han hecho mejoras que solo modifican el operador de selección de nuevas soluciones.De esta forma podemos ver que cambios tienen más impacto sobre el algoritmo.

3.1 Hibridación con búsqueda local (Solis-Wets)

El ABC clásico es un buen explorador del espacio de búsqueda, pero su capacidad de **refinamiento local** es limitada: la ecuación de selección solo modifica una componente por candidata y depende de las diferencias entre fuentes, por lo que cuando la población converge esas diferencias se hacen pequeñas y el avance se ralentiza. La idea de esta hibridación es convertir el algoritmo en un algoritmo memético, combinando la exploración global del ABC con una búsqueda local que intente refinar con más detalle las mejores soluciones

3.1.1 Búsqueda local: Solis-Wets

Solis-Wets es una búsqueda local estocástica de paso adaptativo que opera sobre **una única solución**. En lugar de explorar todo el espacio, da pequeños pasos aleatorios alrededor de la solución actual y aprende en qué dirección conviene moverse mediante dos elementos:

- Un **paso** δ que controla el tamaño de las perturbaciones y que se ajusta de forma dinámica según el algoritmo vaya teniendo éxito o no.
- Un vector de **sesgo** que acumula la dirección de los movimientos que han funcionado, de modo que la búsqueda tiende a continuar avanzando hacia donde ya ha mejorado.

En cada iteración se genera una perturbación aleatoria y se construye una solución candidata sumando el paso y el sesgo:

$$x' = x + d + \text{bias}$$

Si la candidata mejora, se acepta y se refuerza el sesgo en esa dirección. Si no mejora, se prueba el movimiento opuesto $x' = x - d - \text{bias}$,si este mejora, se acepta y el sesgo se ajusta . Si ninguno de los dos mejora, se considera un fracaso y el sesgo se reduce a la mitad para no volver a tomar una dirección que ya no es útil.

El paso δ se adapta para equilibrar exploración y precisión: tras 5 éxitos consecutivos se duplica y tras 3 fracasos consecutivos se divide entre dos . Esto permite que la BL dé pasos grandes cuando está lejos del óptimo local y pasos finos cuando está cerca de él.

3.1.2 Integración en el ABC

La búsqueda local no sustituye a ninguna fase del ABC, sino que se intercala dentro del bucle principal. La hibridación se controla con tres parámetros:

- **ls_freq**: cada cuántos ciclos del ABC se aplica la búsqueda local.
- **ls_budget**: número de evaluaciones que se usan en cada aplicación de la BL.
- **ls_delta**: paso inicial δ de Solis-Wets.

Al final de cada ciclo, y solo una vez cada ls_freq ciclos, se localiza la mejor fuente de la población actual y se le aplica Solis-Wets con unas determinadas evaluaciones. Después se reinicia su contador de trials ya que como acaba de ser mejorada , no tiene sentido considerarla agotada y queremos que se intente explotar todavía más. Las evaluaciones que consume la BL se descuentan del presupuesto global del algoritmo, de forma que ABC y BL comparten el mismo límite de evaluaciones.

```
# dentro del bucle principal del ABC, al final de cada ciclo:
cycle += 1
si use_ls y cycle % ls_freq == 0 y evals < maxevals:
  best = indice de la fuente con menor obj    # la mejor
  budget = min(ls_budget, maxevals - evals)   # no pasarse del limite
  evals += soliswets(foods[best], obj[best], ls_delta, budget, ...)
  trial[best] = 0                             # recien intensificada
  actualizar_mejor(foods[best])
```

3.2 Hibridación con Differential Evolution

Differential Evolution es una metaheurística poblacional que genera nuevas soluciones combinando las diferencias entre individuos de la población. Su operador básico es la mutación diferencial: a un vector base se le suma la diferencia escalada entre otros dos individuos, $F \cdot (x_{r2} - x_{r3})$, donde F es un factor de escala. Después, un cruce binomial mezcla el mutante con la solución original copiando cada componente con probabilidad CR, garantizando con un índice aleatorio que al menos una componente provenga del mutante. Finalmente se aplica una selección *greedy* entre la candidata y la original. El operador de DE combina información de toda la población en varias dimensiones a la vez, lo que suele dar un mejor poder de búsqueda. Se han probado dos formas distintas de incorporar DE al ABC.

3.2.1 Hibridación completa con DE

Esta hibridación sigue exactamente el mismo esquema que la hibridación con Solis-Wets:intercala el algoritmo DE completo dentro del bucle del ABC. El DE Recorre toda la población aplicando a cada individuo mutación, cruce y selección greedy y repite el proceso hasta agotar su presupuesto de evaluaciones. La mutación que se usa parte de un vector base aleatorio :

$$v_{ij} = x_{r1j} + F \cdot \left(x_{r2j} - x_{r3j}\right)$$

con $r1, r2, r3$, i tres índices distintos entre sí.

El DE se ejecuta sobre las fuentes del ABC, las modifica in place y devuelve el número de evaluaciones consumidas para que el ABC las tenga en cuenta. Se controla con los mismos tres parámetros:

- **de_freq**: cada cuántos ciclos del ABC se lanza el DE.
- **de_budget**: evaluaciones concedidas a cada ejecución del DE.
- **F, CR**: parámetros del DE.

```
# algoritmo DE completo, ejecutado sobre la poblacion del ABC:
funcion differential_evolution(foods, obj, trial, F, CR, maxevals):
  evals = 0
  mientras evals < maxevals:
    para i en 0..SN:
      r1, r2, r3 = tres indices distintos de i y entre si
      jrand = dimension aleatoria
      cand = copia(foods[i])
      para j en 0..dim:
        si j == jrand o rand() < CR:                    # cruce binomial
          cand[j] = foods[r1][j] + F*(foods[r2][j]-foods[r3][j])
      candfit = fitness(clip(cand));  evals += 1
      si candfit < obj[i]:                              # seleccion greedy
        foods[i] = cand;  obj[i] = candfit;  trial[i] = 0
  devolver evals

# dentro del bucle principal del ABC, al final de cada ciclo:
cycle += 1
si use_de y cycle % de_freq == 0 y evals < maxevals:
  budget = min(de_budget, maxevals - evals)
  evals += differential_evolution(foods, obj, trial, F, CR, budget)
```

3.2.2 Uso del operador de DE para generación de candidatos

En esta variante DE no se añade como una fase nueva, sino que reemplaza la regla de movimiento de las abejas, ese decir usaremos su operador de mutación para encontrar nuevas soluciones. Donde el ABC clásico usaba la ecuación vista anteriormente para generar nuevas soluciones, ahora se usa el operador del Diferential Evolution con cruce binomial.En este caso para que tenga más efecto se usa en vez de una solución aleatoria la mejor solución, para intentar crear soluciones en esa dirección. Para cada componente usamos el cruce:

$$v_{ij} = x_{ij} + F \cdot \left(x_{\text{best } j} - x_{ij}\right) + F \cdot \left(x_{r1j} - x_{r2j}\right)$$

donde x_{best} es la mejor fuente actual del enjambre y $r1, r2$ son dos índices aleatorios distintos entre sí, de i y de *best*. El término $F \cdot (x_{\text{best}} - x_i)$ mueve la solucion escogida hacia la mejor solución, a diferencia del ABC normal que se mueve solo en la dirección de un vecino escogido aleatoriamente que puede no ser la mejor solución, y $F \cdot (x_{r1} - x_{r2})$ añade una perturbación diferencial que mantiene la diversidad. El cruce binomial con probabilidad CR decide qué componentes se mutan y asegura que al menos una cambie.

Este operador se usa la fase de las abejas obreras y en la de las abejas observadoras del ABC de esta manera el esquema del algoritmo se mantiene y la única diferencia con el ABC original es cómo se genera cada candidata. Los parámetros usados son $F = 0.5$ y $CR = 0.9$, valores habituales en DE.

```
# reemplaza a produceCandidate (ec. 2.2) en empleadas y observadoras:
funcion produceCandidate_DE(foods, i):
  best = indice de la mejor fuente
  r1, r2 = dos indices aleatorios distintos de i, best y entre si
  jrand = dimension aleatoria
  cand = copia(foods[i])
  para j en 0..dim:
    si j == jrand o rand() < CR:                          # cruce binomial
      cand[j] = foods[i][j] + F*(foods[best][j]-foods[i][j])
                            + F*(foods[r1][j]-foods[r2][j])
  devolver clip(cand)
```

3.3 Comparación de las hibridaciones

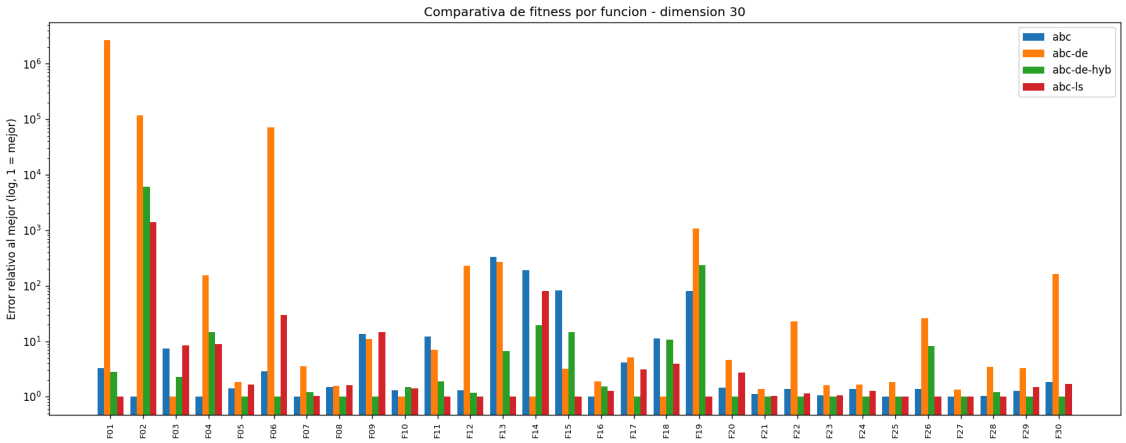


Figura 4: Comparación error para dimensión 30

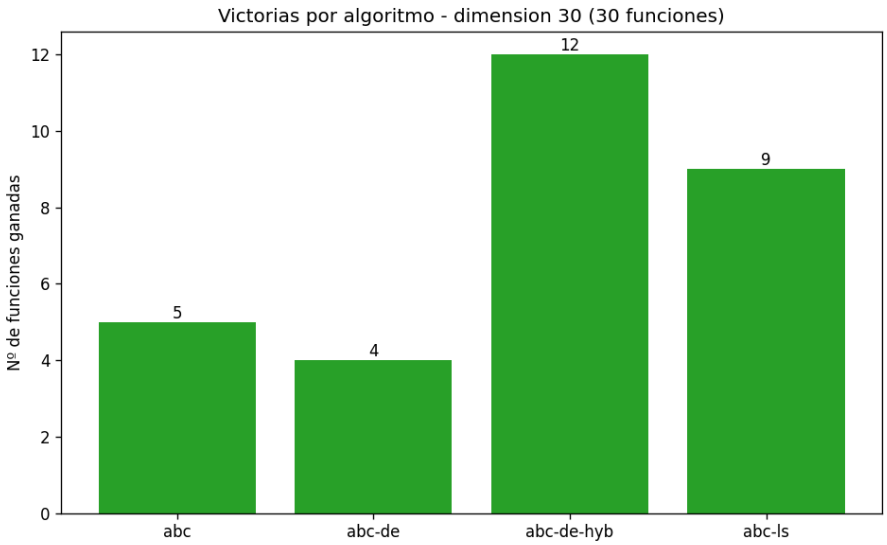


Figura 5: Número de victorias dimensión 30

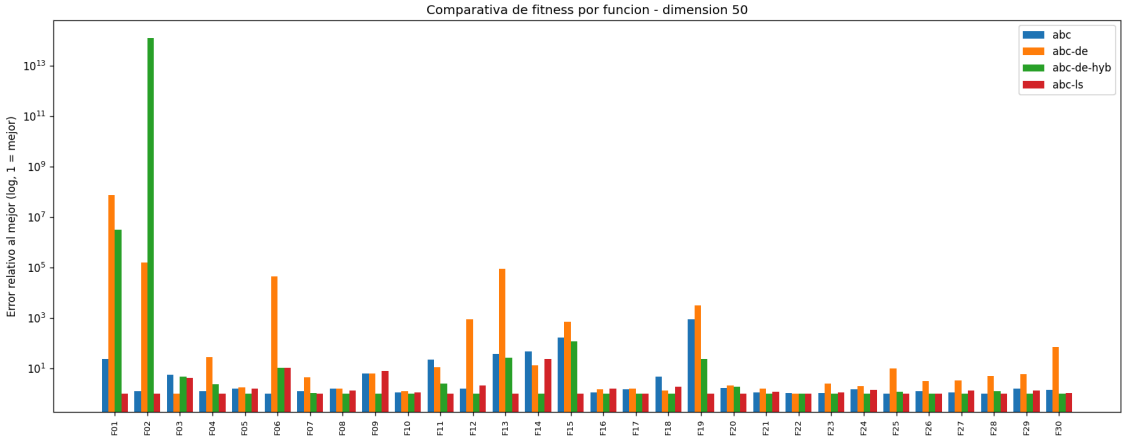


Figura 6: Comparación error para dimensión 50

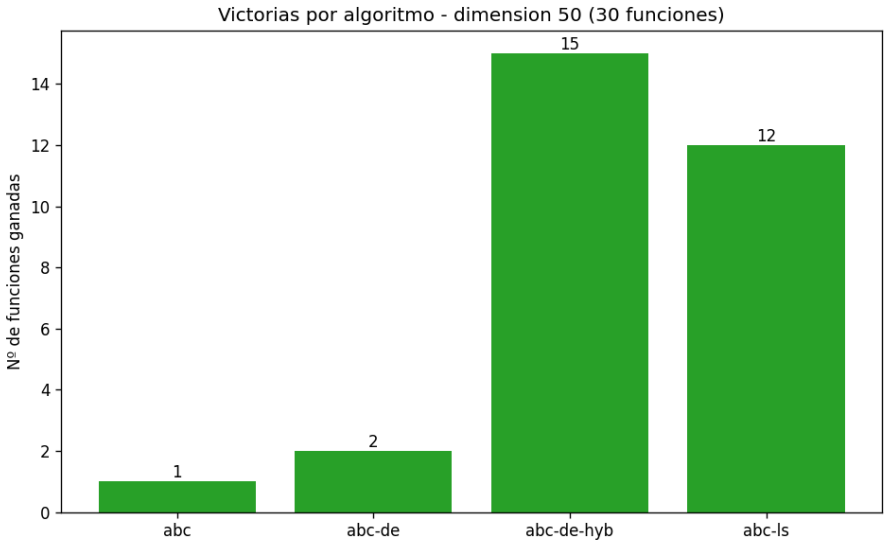


Figura 7: Número de victorias dimensión 50

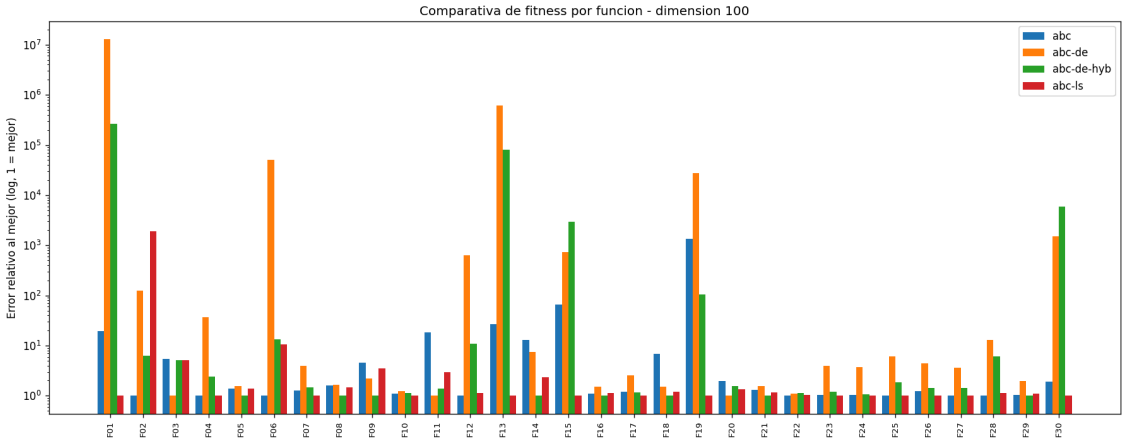


Figura 8: Comparación error para dimensión 100

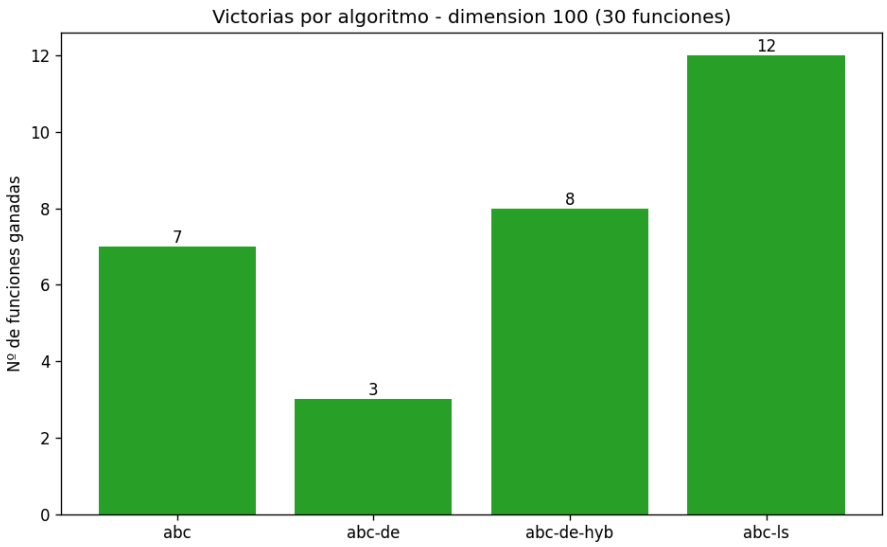


Figura 9: Número de victorias dimensión 100

4 Propuestas de mejora

Uno de los puntos débiles del ABC es la forma en las que se explotan las funetes de alimento ya que al tomar vecinos aleatoriamente para generar nuevas soluciones , podemos estar tomando soluciones malas y estar moviendo nuestra solución hacia soluciones peores.

Esto hace que la convergencia del algoritmo sea lenta y que no se puedan alcanzar las mejores soluciones dentro de las evaluaciones permitidas.

4.1 ABC guiado por la mejor solución global Gbest-guided ABC

Una de las mejoras que se pueden aplicar consiste en cambiar la forma en la que las abejas generan nuevas soluciones. En el ABC clásico la dirección en la que se modifican las soluciones es local ya que la perturbación $\phi_{ij} \cdot (x_{ij} - x_{kj})$ desplaza la fuente i en la dirección de otra fuente k elegida al azar, sin tener en cuenta dónde se encuentra la mejor solución conocida. Esto favorece la exploración, pero ralentiza la convergencia ya que la información de la mejor solución encontrada no se aprovecha para guiar la búsqueda.

El Gbest-guided ABC , corrige esto inspirándose en la actualización de velocidad del PSO, añade a la ecuación de movimiento un término adicional que atrae cada nueva fuente hacia la **mejor solución global** encontrada hasta el momento.Así el algoritmo conserva la componente exploratoria del ABC y aumenta la presión de explotación que acelera la convergencia hacia las regiones prometedoras del espacio de búsqueda.

4.1.1 Regla de generación del candidato

La regla de generación del candidato se reemplaza por una expresión con dos términos de perturbación. Sobre la componente j elegida al azar:

$$v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj}) + \psi_{ij} \cdot (y_j - x_{ij})$$

donde y es la mejor solución global encontrada hasta el momento y cada término hace lo siguiente:

- El término $\phi_{ij} \cdot (x_{ij} - x_{kj})$, con $\phi_{ij} \in [-1, 1]$ y $k \neq i$ una fuente aleatoria, es el movimiento clásico del ABC y aporta la exploración: genera diversidad usando las soluciones vecinas.
- El término $\psi_{ij} \cdot (y_j - x_{ij})$ es la parte que arrastra la fuente i hacia la mejor solución global, de forma proporcional a la distancia que las separa, lo que acelera la convergencia.
- El factor ψ_{ij} es un número aleatorio en $[0, C]$, donde C es una constante positiva que controla el peso de la atracción hacia la mejor solución. Un valor de C demasiado grande provoca una convergencia prematura, mientras que uno demasiado pequeño desaprovecha la guía global.

Lo mejor de esta modificación es que mantiene el mismo esquema que el ABC original y solo tenemos que cambiar la función del operador que genera nuevas soluciones.

4.2 ABC guiado por gbest con límite de dinámico

El término gbest acelera mucho la convergencia, pero tiene un efecto secundario: como todas las fuentes son atraídas hacia la misma solución global, el enjambre tiende a *colapsar* en un único punto. Cuando esto ocurre, el ABC deja de progresar, porque la perturbación clásica $\phi_{ij} \cdot (x_{ij} - x_{kj})$ es proporcional a la distancia entre fuentes: si todas las fuentes están casi en el mismo sitio, los movimientos que generan son pequeños y la búsqueda se estanca.

El único mecanismo del ABC que introduce diversidad real es la *abeja exploradora*, que reinicia una fuente con un salto aleatorio en todo el dominio. Sin embargo, en el ABC clásico esa fase depende de un parámetro limit fijo, una fuente solo se abandona tras limit ciclos sin mejorar. Si limit es muy alto, las fuentes estancadas tardan demasiado en reiniciarse y el enjambre permanece colapsado, si es muy bajo, se reinician fuentes que todavía eran útiles y se pierde capacidad de explotación.

La propuesta consiste en hacer que ese límite **deje de ser fijo y se adapte a la diversidad del enjambre**. Como base para esta modificación voy a seleccionar el ABC que usa Gbest ya que es el que mejor funciona y lo único que cambio es cuándo se reinicia una fuente.

4.2.1 Medida de diversidad

En cada ciclo se mide la dispersión de las fuentes calculando, para cada dimensión j, la desviación típica σ_j de las SN fuentes, promediando sobre las n dimensiones y normalizando por el tamaño del dominio:

$$D = \frac{1}{n} \sum_{j=1}^{n} \frac{\sigma_j}{\text{upper} - \text{lower}}$$

El valor D es cercano a 0 cuando el enjambre está colapsado y máximo cuando está disperso. Esta medida no consume evaluaciones, ya que solo usa las coordenadas de las fuentes que ya tenemos.

4.2.2 Límite dinámico

El límite de abandono se interpola entre un mínimo y un máximo en función de la diversidad actual respecto a la inicial D_0 (la de la población aleatoria de partida, que es la más dispersa). Definiendo $\text{frac} = D/D_0$ recortado a $[0, 1]$, el límite del ciclo es:

$$\text{limit} = \text{limit}_{\min} + \text{frac} \cdot (\text{limit}_{\max} - \text{limit}_{\min})$$

El comportamiento resultante es:

- **Poca diversidad** (enjambre colapsado, $D \approx 0$): el límite baja hacia $\text{limit}_{\min}$, de modo que las fuentes estancadas se reinician antes y se inyecta diversidad mediante saltos aleatorios.
- **Mucha diversidad** (enjambre disperso, $D \approx D_0$): el límite sube hacia $\text{limit}_{\max}$, dejando que las fuentes exploten más tiempo hacia la mejor solución global.

De esta forma se obtiene un lazo de realimentación que regula automáticamente el equilibrio exploración-explotación: cuando el enjambre converge demasiado, el propio algoritmo aumenta el ritmo de reinicios para reabrir la búsqueda, y cuando hay diversidad suficiente, relaja los reinicios para afinar la solución. A diferencia de una estrategia que simplemente desactivase la guía global cuando baja la diversidad, aquí la diversidad se recupera con el reinicio aleatorio de la exploradora, cuyo salto es de magnitud grande e independiente de lo colapsado que esté el enjambre, por lo que sí consigue reabrir la búsqueda de manera efectiva.

Los parámetros usados son $\text{limit}_{\min} = 20$ y $\text{limit}_{\max} = 200$, manteniendo $C = 1.5$ para el término gbest.

4.3 Comparación de resultados

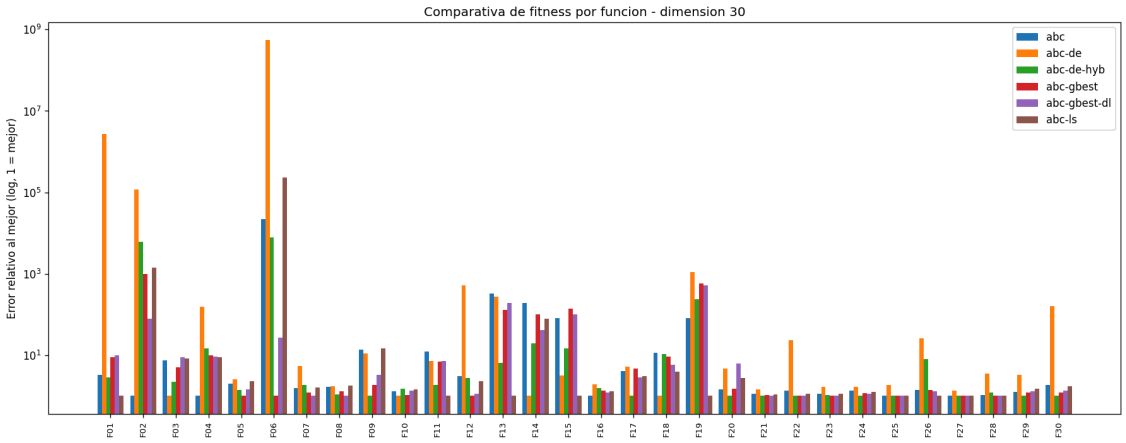


Figura 10: Comparación error para dimensión 30

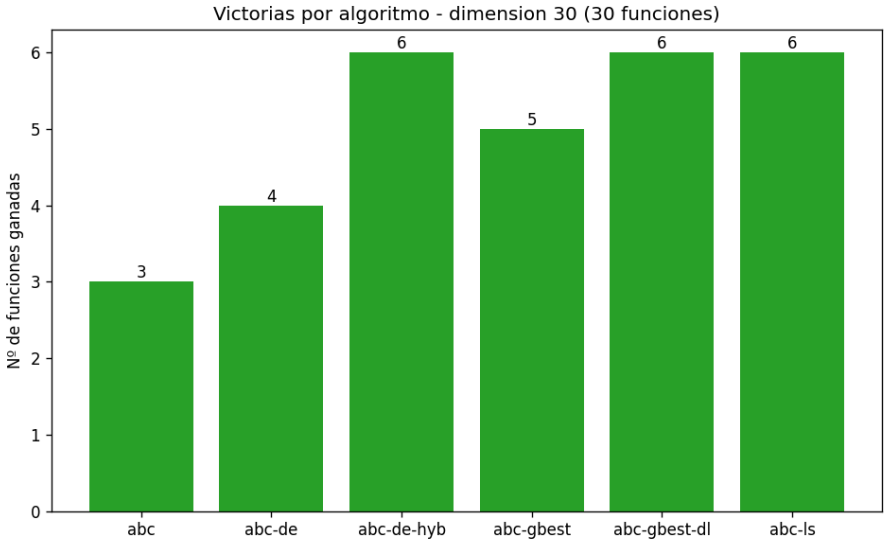


Figura 11: Número de victorias dimensión 30

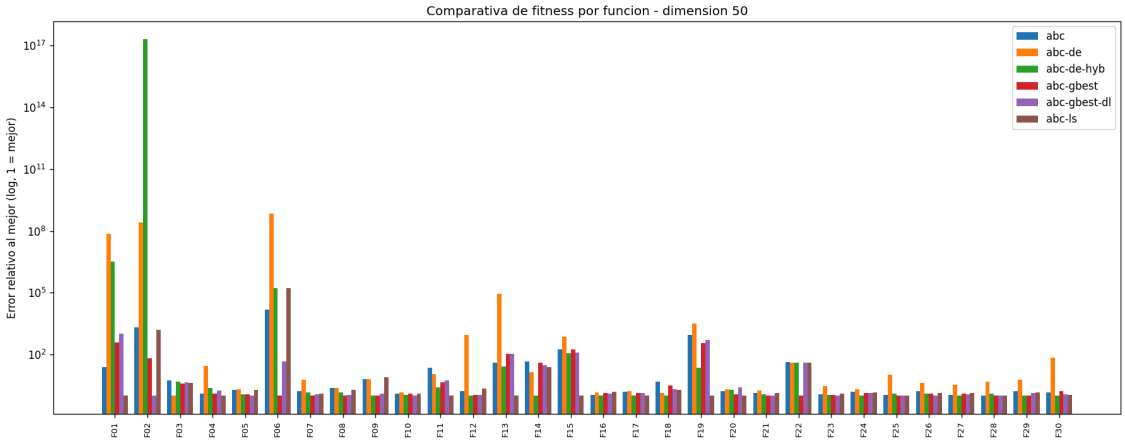


Figura 12: Comparación error para dimensión 50

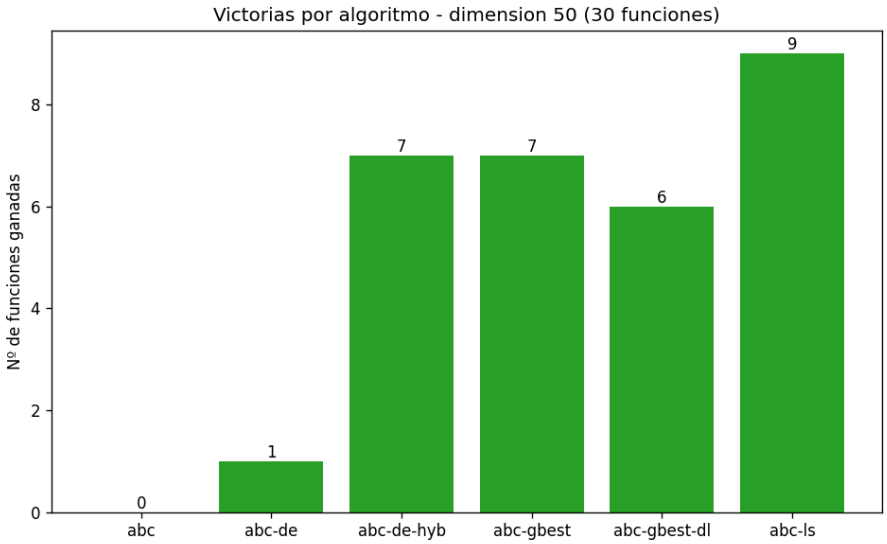


Figura 13: Número de victorias dimensión 50

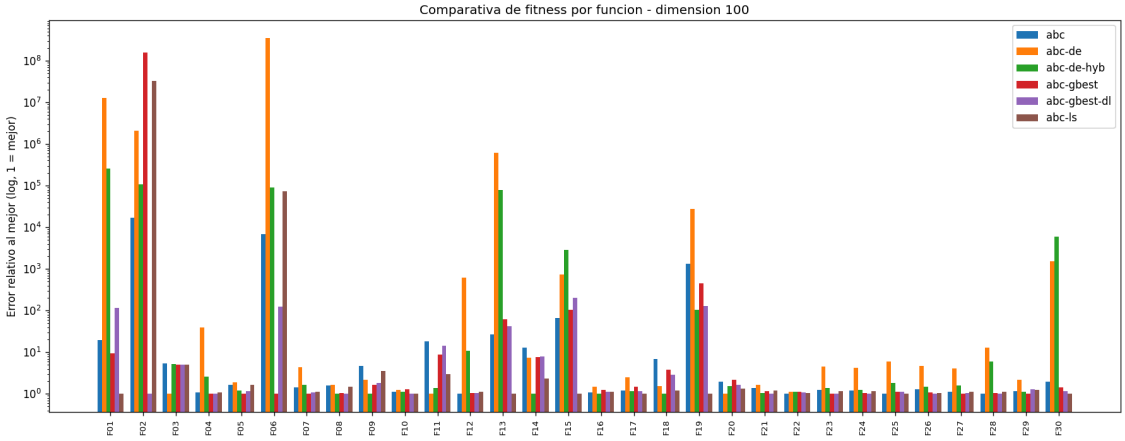


Figura 14: Comparación error para dimensión 100

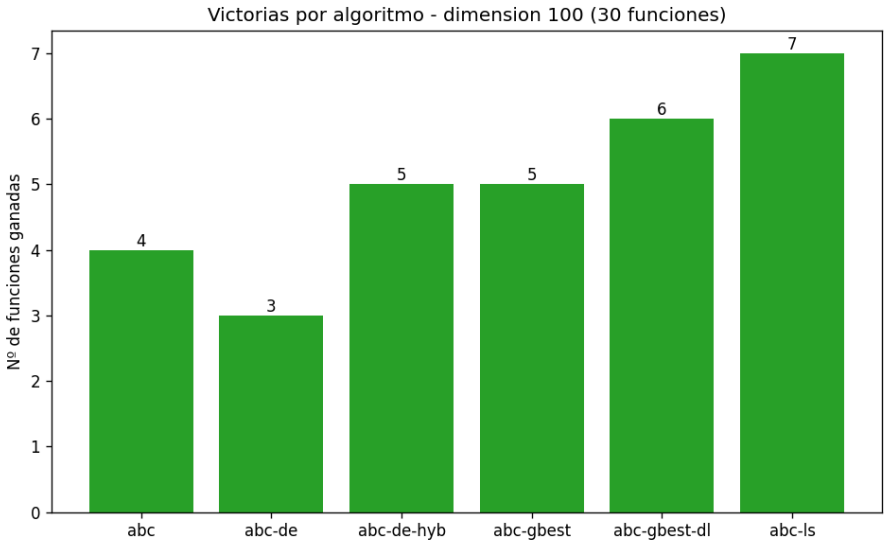


Figura 15: Número de victorias dimensión 100